

Cours de bases de données,
aspects systèmes,
<http://sys.bdpedia.fr>

Les itérateurs

Plan d'exécution et opérateurs

Rappel : un **plan d'exécution** est un arbre constitué **d'opérateurs** échangeant des **flux de données**.

Plan d'exécution et opérateurs

Rappel : un **plan d'exécution** est un arbre constitué **d'opérateurs** échangeant des **flux de données**.

Caractéristiques des opérateurs

- ont une forme générique (**itérateurs**) ;
- fournissent une tâche spécialisée (cf. l'algèbre relationnelle)
- peuvent être ou non **bloquants**.

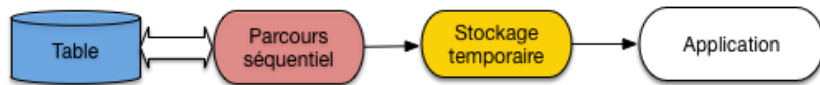
Un petit nombre suffit pour couvrir SQL !

Dans ce cours

On va apprendre à implanter un moteur d'exécution pour (presque toutes) les requêtes SQL !

Mode naïf : matérialisation

Dans ce mode, un opérateur calcule son résultat, puis le transmet.

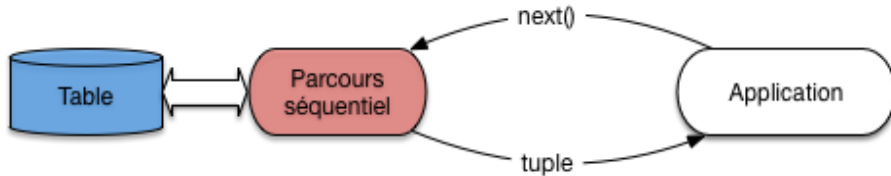


Deux inconvénients :

- Consomme de la mémoire.
- Introduit un temps de **latence**.

La bonne solution : pipelining

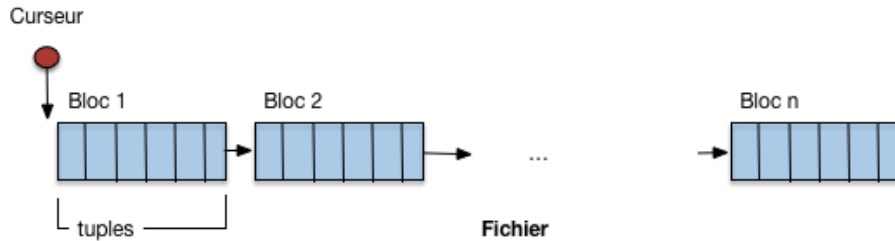
Le résultat est produit **à la demande**.



- Plus de stockage intermédiaire.
- Latence minimale.

Illustration : l'opérateur FullScan

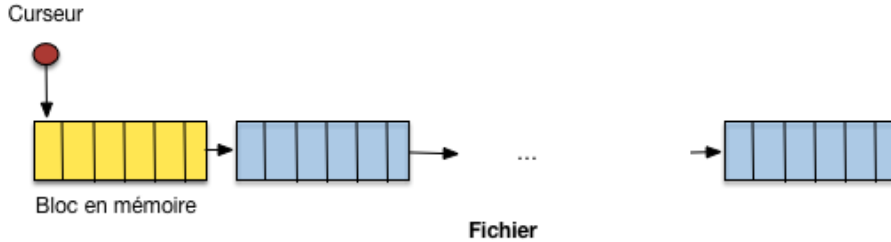
Au moment du `open()`, le curseur est positionné **avant** le premier nuplet.



`open()` désigne la phase d'initialisation de l'opérateur.

Illustration : l'opérateur FullScan

Le premier `next()` entraîne l'accès au premier bloc, placé en mémoire.



Le curseur se place sur le premier nuplet, qui est retourné comme résultat. **Le temps de réponse est minimal.**

Illustration : l'opérateur FullScan

Le deuxième `next()` avance d'un cran dans le parcours du bloc.

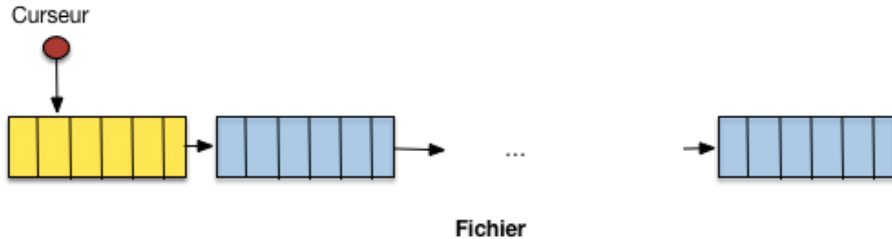


Illustration : l'opérateur FullScan

Après plusieurs `next()`, le curseur est positionné sur le dernier nuplet du bloc.

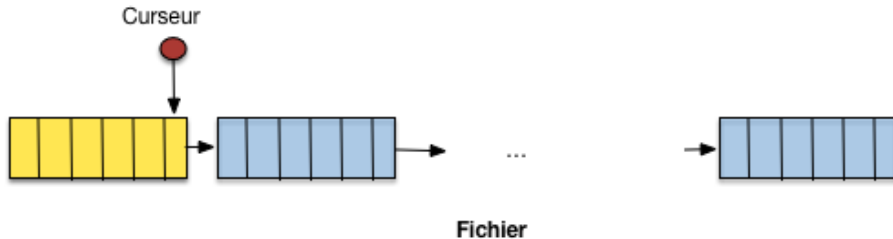
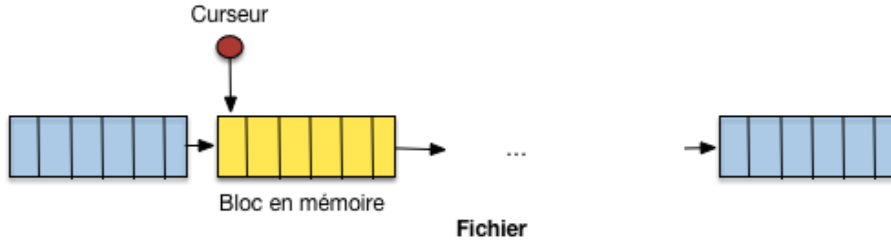


Illustration : l'opérateur FullScan

L'appel suivant à `next()` charge le second bloc en mémoire.



Bilan : besoin en mémoire réduit (1 bloc); temps de réponse très court.

Opérateur bloquant

Tous les opérateurs peuvent-ils fonctionner en mode pipelining ?

```
| select min(date) from T
```

On ne peut pas produire un nuplet avant d'avoir examiné **toute** la table.

Il faut alors introduire un opérateur **bloquant**, avec une latence forte.

Avec un opérateur bloquant, on **additionne** le temps d'exécution et le temps de traitement.

Résumé : opérateurs d'exécution

Principes essentiels :

1. **Itération** : dans tous les cas, un opérateur produit les nuplets à la demande.
2. **Pipelining** : si possible, le résultat est calculé au fur et à mesure.
3. **Matérialisation** : parfois le résultat intermédiaire doit être calculé et stocké.

Bien distinguer

- **Temps de réponse** : temps pour obtenir le premier nuplet.
- **Temps d'exécution** : temps pour obtenir tous les nuplets.

Résumé : opérateurs d'exécution

Principes essentiels :

1. **Itération** : dans tous les cas, un opérateur produit les nuplets à la demande.
2. **Pipelining** : si possible, le résultat est calculé au fur et à mesure.
3. **Matérialisation** : parfois le résultat intermédiaire doit être calculé et stocké.

Bien distinguer

- **Temps de réponse** : temps pour obtenir le premier nuplet.
- **Temps d'exécution** : temps pour obtenir tous les nuplets.

Merci !

Cours de bases de données,
aspects systèmes,
<http://sys.bdpedia.fr>

Exécution et optimisation

Opérateur = itérateur

Tout opérateur est implanté sous forme d'un **itérateur**. Trois fonctions :

- `open` : initialise les ressources et positionne le curseur ;
- `next` : ramène l'enregistrement courant se place sur l'enregistrement suivant ;
- `close` : libère les ressources ;

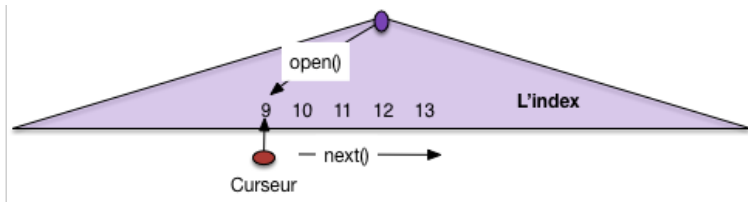
Échanges :

- Un itérateur **consomme** des nuplets d'autres itérateurs **source**.
- Un itérateur **produit** des nuplets pour un autre itérateur (ou pour l'application).

Exemple : parcours d'index (IndexScan)

Rappel : index = arbre B.

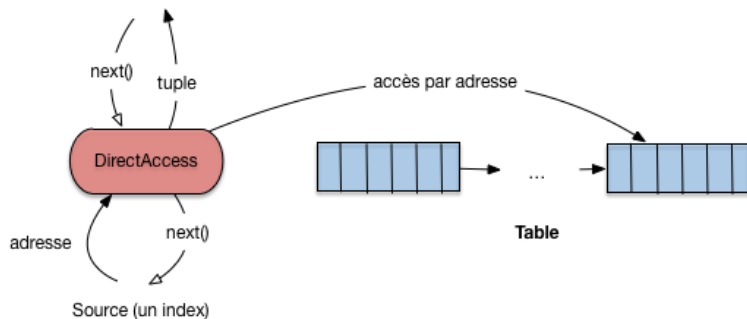
- Pendant le `open()` : parcours de la racine vers la feuille.
- À chaque appel à `next()` : parcours en séquence des feuilles.



Efficacité : très efficace, quelques lectures logiques (index en mémoire)

Accès par adresse : DirectAccess

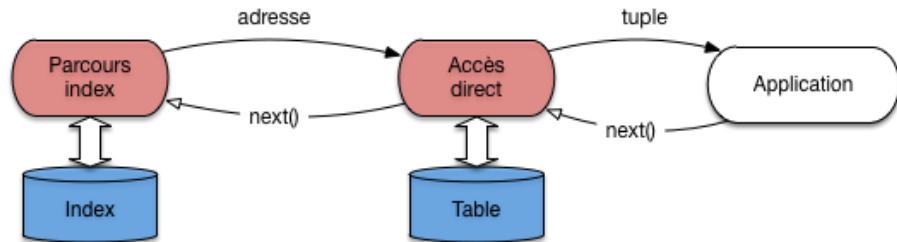
- Pendant le `open()` : rien à faire.
- À chaque appel à `next()` : on reçoit une adresse, on produit un nuplet.



Très efficace : un accès bloc, souvent en mémoire.

Plan d'exécution

Un plan d'exécution connecte les opérateurs. Ici, recherche avec index.



Le pipelining reste complet.

Un exemple de base

Nous allons étudier les plans permettant d'exécuter les requêtes **mono-table**.

```
select a1, a2, ..., an  
from T  
where condition
```

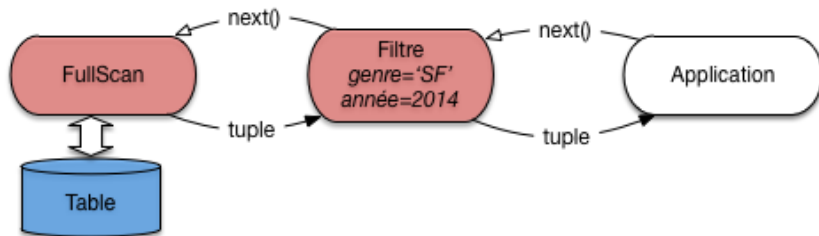
De quels opérateurs a-t-on besoin ?

- [FullScan] : parcours séquentiel de la table (déjà vu).
- [IndexScan] : parcours d'un index (si disponible).
- [DirectAccess] : accès **par adresse** à un nuplet.
- [Filter] : test de la condition.

Nous obtenons **deux** plans d'exécution possibles.

Premier plan d'exécution : sans index

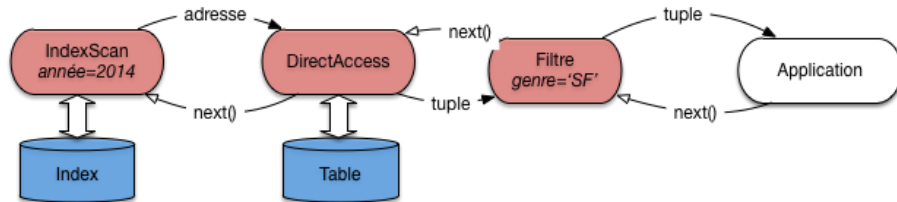
```
| select titre from Film where genre='SF' and annee = 2014
```



N'utilise pas d'index

Second plan d'exécution : avec index

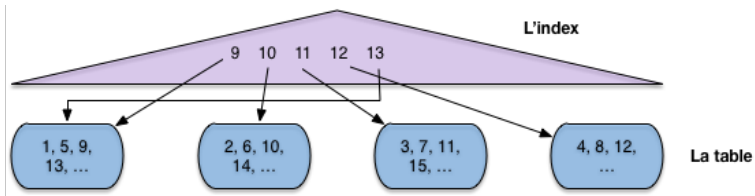
```
| select titre from Film where genre='SF' and annee = 2014
```



Utilise un index sur l'année.

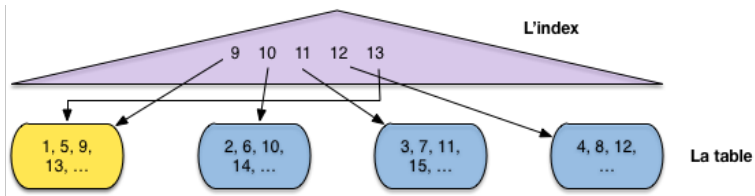
Index ou pas index ?

Recherche des nuplets entre 9 et 13, avec 3 blocs en mémoire.



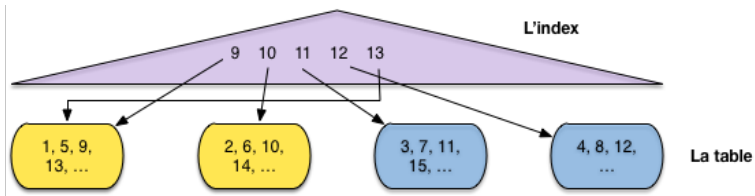
Index ou pas index ?

Recherche des nuplets entre 9 et 13, avec 3 blocs en mémoire.



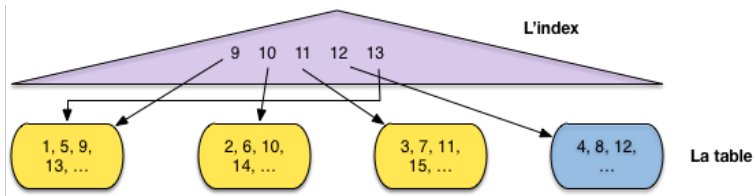
Index ou pas index ?

Recherche des nuplets entre 9 et 13, avec 3 blocs en mémoire.



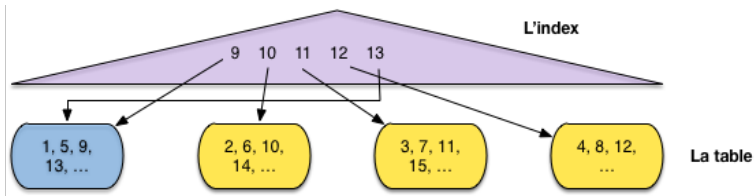
Index ou pas index ?

Recherche des nuplets entre 9 et 13, avec 3 blocs en mémoire.



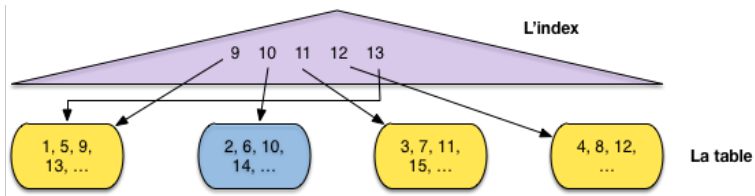
Index ou pas index ?

Recherche des nuplets entre 9 et 13, avec 3 blocs en mémoire.



Index ou pas index ?

Recherche des nuplets entre 9 et 13, avec 3 blocs en mémoire.



Index ou pas index ?

Recherche des nuplets entre 9 et 13, avec 3 blocs en mémoire.

Avec l'index, il faut lire 5 blocs ! Parcours séquentiel bien préférable.

Un cas extrême ! **En pratique** : le SGBD décide en fonction des statistiques et des ressources disponibles.

Opérateur de sélection

Très simple : filtre les nuplets fournis par la source.

```
function nextFilter
{
  # On prend un nuplet de la source
  $nuplet = $source.next();
  # On continue tant que la condition n'est pas satisfaite,
  # ou la source parcourue
  while ($nuplet != null and $nuplet.test($C) = false)
  do
    $nuplet = $source.next();
  done

  return $nuplet;
}
```

Pour compléter

Un plan à exécuter avec index.

```
select * from Film
where idFilm = 20
and titre = 'Vertigo'
```

Un plan à exécuter sans index.

```
select * from Film
where idFilm = 20
or titre = 'Vertigo'
```

Résumé : plans pour requêtes mono-table

Premier aperçu de l'optimisation

- Le système a le choix entre plusieurs plans possibles.
- Distinguer le plus efficace n'est pas toujours trivial.
- Le choix peut changer selon le contexte.

Résumé : plans pour requêtes mono-table

Premier aperçu de l'optimisation

- Le système a le choix entre plusieurs plans possibles.
- Distinguer le plus efficace n'est pas toujours trivial.
- Le choix peut changer selon le contexte.

Merci !

Cours de bases de données,
aspects systèmes,
<http://sys.bdpedia.fr>

Algorithme de tri

Tri externe

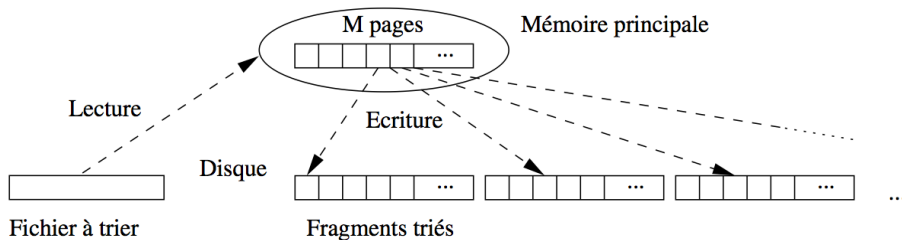
Le **tri externe** est utilisé,

- pour les algorithmes de jointure (*sort/merge*)
- l'élimination des doublons (*clause distinct*)
- pour les opérations de regroupement (*group by*)
- ... et bien sûr pour les *order by*

C'est une opération qui peut être très coûteuse sur de grands jeux de données.

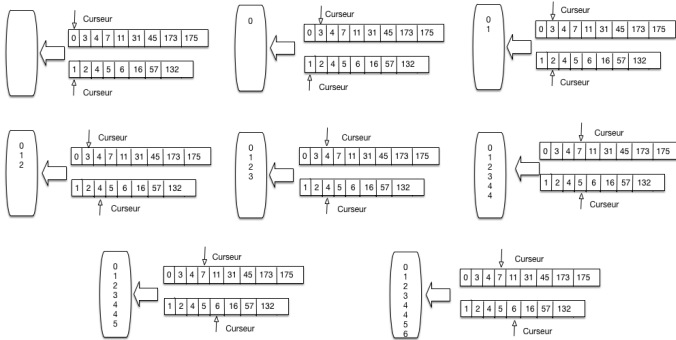
Première phase : le tri

On remplit la mémoire, on trie, on vide dans des **fragments**, et on recommence.



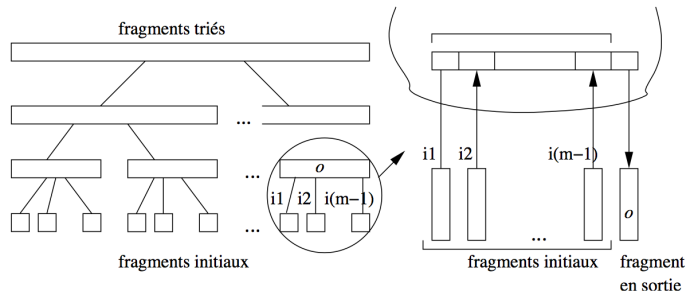
Coût : une lecture + une écriture du fichier.

Fusion de listes triées



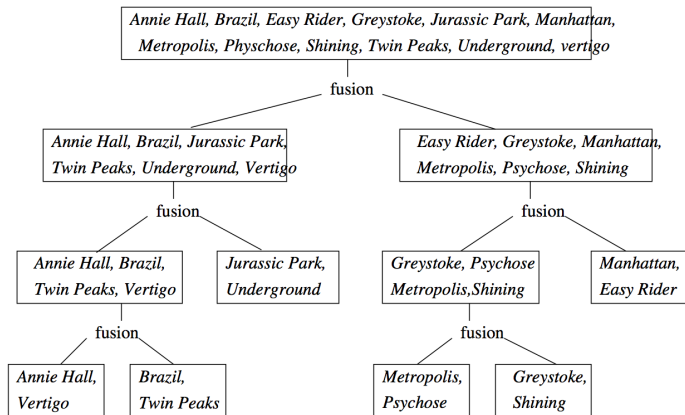
Deuxième phase : la fusion

On groupe les fragments par M (taille de la zone de tri), et on fusionne.



Coût : autant de lectures/écritures du fichier que de niveaux de fusion.

Illustration avec $M = 3$



Essentiel : la taille de la zone de tri

Un fichier de 75 000 pages de 4 Ko, soit 307 Mo.

- $M > 307\text{Mo}$: une lecture, soit 307
- $M = 2\text{Mo}$, soit 500 pages.

le tri donne $\lceil \frac{307}{2} \rceil = 154$ fragments.

On fait la fusion avec 154 pages

Coût total de $614 + 307 = 921$ Mo.

NB : il faut allouer beaucoup de mémoire pour passer de 1 à 0 niveau de tri.

Avec très peu de mémoire

$M = 1\text{Mo}$, soit 250 pages.

- on obtient 307 fragments.
- On fusionne les 249 premiers fragments, puis les 58 restant. On obtient F_1 et F_2 .
- On fusionne F_1 et F_2 .

Coût total : $1\,228 + 307 = 1\,535$ Mo.

Résultat : grosse dégradation entre 2 Mo et 1 Mo (calcul approximatif).

L'opérateur de tri

L'opérateur de tri est **bloquant**

- Les deux phases du tri-fusion sont effectuées pendant le *open()*
- Le *next()* ne fait que lire, un à un, les tuples dans le résultat du tri.

Conséquence : **latence importante des requêtes impliquant un tri**

Grosse différence entre :

```
| select * from Film
```

et

```
| select * from Film order by titre
```

Cours de bases de données,
aspects systèmes,
<http://sys.bdpedia.fr>

Algorithmes de jointure

L'opérateur qui nous manque

La jointure : opération très courante, potentiellement coûteuse.

L'opérateur de jointure complète notre petit catalogue pour (presque) toutes les requêtes SQL.

```
select a1, a2, ..., an  
from T1, T2, ..., Tm  
where T1.x = T2.y and ...  
order by ...
```

Plusieurs algorithmes possibles.

Principaux algorithmes

On va se limiter à quelques exemples représentatifs :

Jointure avec index

- Algorithme de jointure par boucles imbriquées indexées.

Jointure sans index

- Le plus simple : **boucles imbriquées (non indexée)**.
- Plus sophistiqué : la **jointure par hachage**.

Jointure avec index

Très **courant** ; on effectue **naturellement** la jointure sur les clés primaires/étrangères.

- Les films et leur metteur en scène

```
select * from Film as f, Artiste as a  
where f.id_realisateur = a.id
```

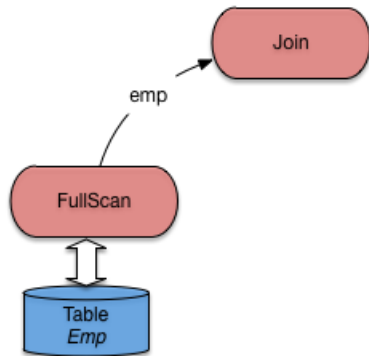
- Les employés et leur département

```
select * from emp e, dept d  
where e.dnum = d.num
```

Garantit **au moins** un index sur la condition de jointure.

Jointure avec index : l'algorithme

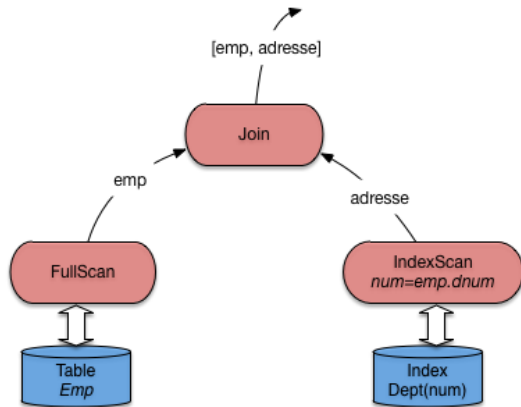
On parcourt séquentiellement la table contenant la clé étrangère.



On obtient des nuplets employé, avec leur no de département.

Jointure avec index : l'algorithme

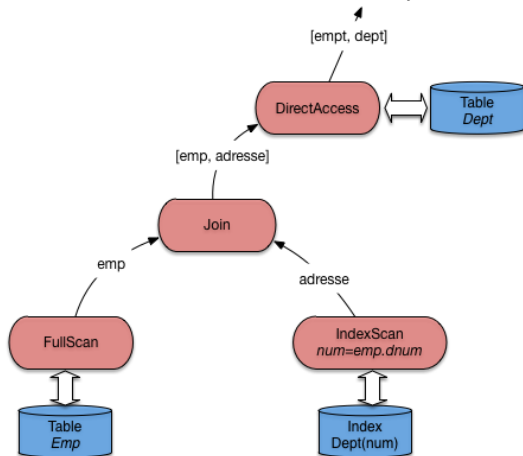
On utilise le no de département pour un accès à l'index Dept.



On obtient des paires [employé, adrDept].

Jointure avec index : l'algorithme

Il reste à résoudre l'adresse du département avec un accès direct.



On obtient des paires [employé, dept].

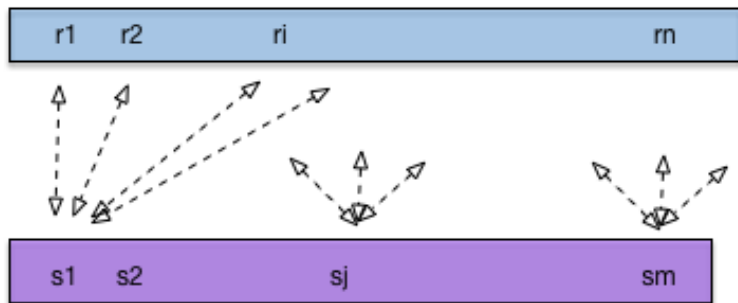
Jointure avec index : l'algorithme

Avantages :

- Efficace (un parcours, plus des recherches par adresse)
- Favorise le temps de réponse et le temps d'exécution

Jointures par boucles imbriquées

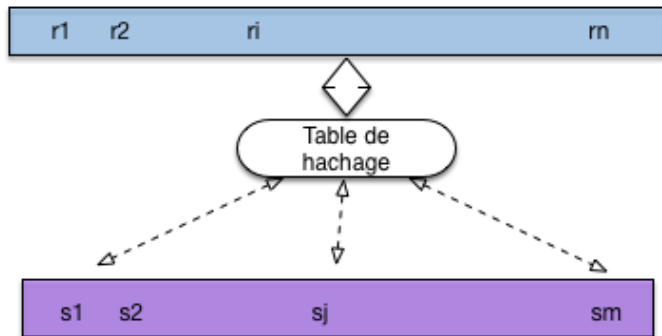
Pas d'index ? La méthode de base est d'énumérer **toutes** les solutions possibles.



Coût quadratique. Acceptable pour deux petites tables.

Jointures par hachage

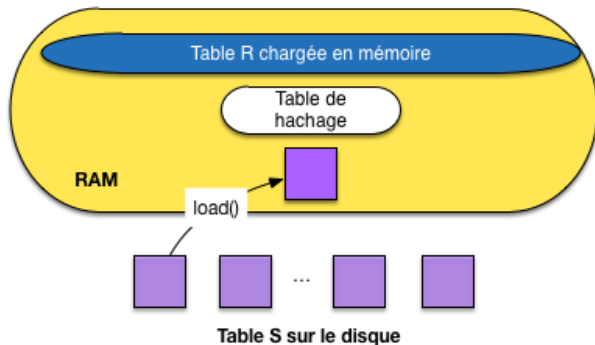
Meilleure solution : construire une table de hachage sur une des tables.



Evite les $O(n^2)$ comparaisons. Appelons cette méthode **JoinList**.

Mémoire insuffisante ?

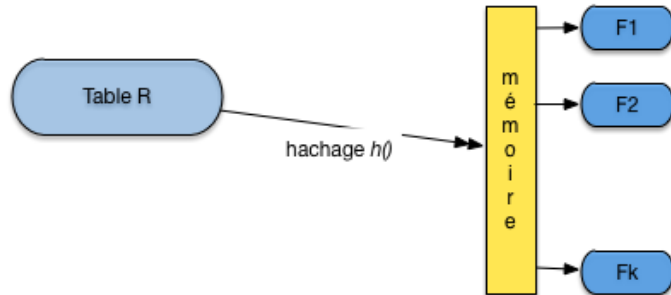
Essayons de placer **une** des deux tables en mémoire.



On charge l'autre bloc par bloc ; on applique **JoinList**.
Une seule lecture de chaque table suffit.

Et quand *aucune* table ne tient en mémoire ?

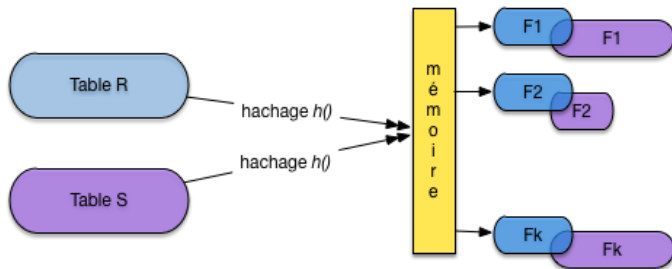
On hache la plus petite des deux tables en k fragments.



Essentiel : les fragments doivent tenir, chacun, en mémoire.

Et quand *aucune* table ne tient en mémoire ?

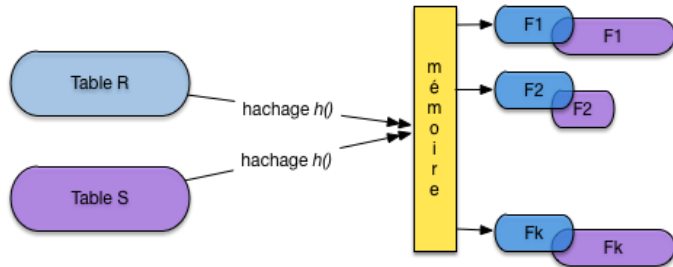
On hache la seconde table, avec la même fonction $h()$, en k autres fragments.



Cette fois, on n'impose pas la contrainte que les fragments tiennent en mémoire.

Et quand *aucune* table ne tient en mémoire ?

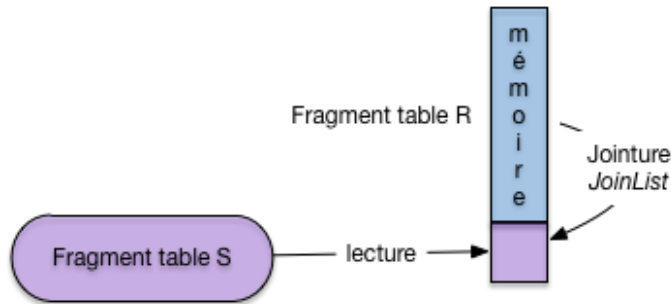
On effectue la jointure sur les paires de fragments $(F_1^R, F_1^S), (F_2^R, F_2^S), (F_k^R, F_k^S)$,



Propriété : Deux nuplets r et s doivent être joints si et seulement si ils sont dans des fragments associés.

Illustration : phase de jointure

On charge F_R^i de R en mémoire ; on parcourt F_S^i de S et on joint.



Déjà vu ? Oui : jointure par boucles imbriquées quand une table tient en mémoire.

Résumé : la jointure

Un opérateur potentiellement coûteux. Quelques principes généraux :

- Si **une** table tient en mémoire : jointure par boucle imbriquées, ou hachage.
- Si **au moins un** index est utilisable : jointure par boucle imbriquées indexée.
- Si une des deux tables beaucoup plus petite que l'autre : jointure par hachage.
- Sinon : jointure par tri-fusion (non présenté).

Décision très complexe, prise par la système en fonction des statistiques.

Résumé : la jointure

Un opérateur potentiellement coûteux. Quelques principes généraux :

- Si **une** table tient en mémoire : jointure par boucle imbriquées, ou hachage.
- Si **au moins un** index est utilisable : jointure par boucle imbriquées indexée.
- Si une des deux tables beaucoup plus petite que l'autre : jointure par hachage.
- Sinon : jointure par tri-fusion (non présenté).

Décision très complexe, prise par la système en fonction des statistiques.

Merci !